Fewest GPUs That Meet the SLO

Binary search on the answer, run on a real production trace

Tracks: Search (binary search on the answer, doubling search, monotone checks) **Code:** `src/FewestGpus/`. **Tests:** 52, run with `dotnet test`. **Demo:** `demo/`, on the Azure LLM inference trace

---

0. The problem

> An LLM endpoint has a latency target on p99 time to first token. Every machine costs money. What is the fewest machines that still meets the target for this traffic?

You can't answer it by dividing traffic by one machine's capacity, because requests queue behind each other. The only way to know if n machines is enough is to run a check (a load test or a simulation). Checks are expensive, so the goal is to run as few as possible.

▼ Hint 1
If 12 machines meet the target, what do you know about 13?

▼ Hint 2
Write the result for n = 1, 2, 3 ... as a row of N and Y. What shape is it?

▼ Hint 3
You don't know a good upper bound. How fast can you find one?

---

1. Background

- An SLO is "a target value or range of values for a service level that is measured by an SLI", and latency targets use percentiles because a high one "shows you a plausible worst-case value" (Google SRE book, ch. 4).
- Time to first token (TTFT) is "the time from query submission to the first received token", and it "generally includes request queuing time, prefill time, and network latency" (NVIDIA NIM docs).
- Benchmark tools search the same way: Triton's Perf Analyzer has a `--binary-search` option over concurrency or request rate, stopped by a `--latency-threshold` (Perf Analyzer CLI).

---

2. The mental model

```
n:      1  2  3  4  5  6  7  8  9  10  11  12  13  14  15  16
meets:  N  N  N  N  N  N  N  N  N  N   Y   Y   Y   Y   Y   Y
```

More machines never make it slower, so the row is always N's, then Y's. The answer is the first Y.

So one check settles a whole side: an N at 6 means 1 to 5 are N, a Y at 12 means 13 to 16 are Y. Check the middle, drop the settled half, repeat. Here: 8 N, 12 Y, 10 N, 11 Y. Four checks.

You aren't searching an array. You're searching the range of possible answers, and each "element" is a check you have to run.

---

3. Three ways to search

Version	Checks for answer A	Needs an upper bound H?
Try 1, 2, 3 ...	A	No
Binary search over 1 to H	about log2(H)	Yes
Doubling (1, 2, 4, 8 ...) until a Y, then binary search between the last N and that Y	about 2 log2(A)	No

Binary search is only as good as its upper bound. Without a tight one you need a safe one, and a safe one can be huge. Section 5 shows what that costs on real traffic.

---

4. Why it's correct

The search. At every step, everything below `lo` is known N, everything above `hi` is known Y, and `best` is the smallest Y seen. The range halves each step, so it ends. At the end, `best` was checked and passed, and `best - 1` was checked and failed (unless `best` is the start). A test checks both are in the log.

The check must be monotone, or binary search returns a wrong number and nothing looks broken. For the simulator here (one first-come, first-served queue, each request to the first free machine), adding a machine never makes any request later:

1. Treat n machines as n + 1 where one is never free. Sort each side's "free at" times.
2. At the start, the side with the real extra machine is no later in every position.
3. Each request takes the earliest free time on each side, so it starts no later, and the slot it fills is no later. Sorting keeps the order position by position.

So every request's TTFT is no larger, and so is any percentile of it. Tests check this request by request on random traces.

---

5. On real traffic: the Azure LLM inference trace

Inputs, and where each comes from:

Input	Value	Source
Traffic	Two one-hour traces from Azure LLM inference services, coding (8,819 requests) and conversation (19,366), replayed as recorded	Azure LLM Inference Dataset 2023, CC-BY
Target	p99 of (TTFT / the same request's TTFT on an idle DGX-A100) at most 6, for A100 and H100 alike	Splitwise, Table VI (Patel et al., ISCA 2024)

Input	Value	Source
One machine	Llama-70B on one DGX machine, 8 GPUs	Splitwise
Idle TTFT	A straight line in prompt tokens, through Splitwise's Table IV medians (see below)	**Our fit**

The idle TTFT line. Splitwise reports that TTFT "grows almost linearly as the prompt size increases", and its Table IV gives median TTFT without batching: coding 185 ms (A100) and 95 ms (H100), conversation 155 ms and 84 ms. The traces' median prompts are 1,469 and 1,020 tokens. Through those two points:

- A100: (185 - 155) / (1469 - 1020) = 0.0668 ms per token, plus 155 - 1020 x 0.0668 = 86.8 ms fixed
- H100: (95 - 84) / 449 = 0.0245 ms per token, plus 59.0 ms fixed

Two points always fit a line exactly, so this is an estimate, not a validated model.

Results (from `demo/`, reproduce with section 8):

Trace	GPU	Fewest machines	GPUs	Checks: try 1, 2, 3	Checks: binary over 1 to requests	Checks: doubling
Coding	A100	11	88	11	13	8
Coding	H100	5	40	5	13	6
Conversation	A100	2	16	2	14	2
Conversation	H100	1	8	1	14	1

Two things the real trace shows:

- **Size for the bursts, not the average.** Spread evenly, the coding traffic would keep 0.57 A100 machines busy. It needs 11, because it comes in bursts: 67 requests in its busiest second (18:31:26), against an average of 2.57 a second.
- **A safe upper bound can make binary search worse than trying every n.** With the number of requests as the bound, binary search took 13 to 14 checks, more than trying 1, 2, 3 in every row.
- **Doubling isn't free either.** For small answers, trying in order is hard to beat (coding on H100: 5 checks in order, 6 doubling). It pays off as the answer grows (coding on A100: 8 against 11).

---

6. Assumptions and limits

	Status
Search finds the fewest n; check counts in section 3	**Tested** (52 tests, incl. thousands of random cases against trying every n)
Simulator is monotone	**Tested** on random traces, argued in section 4
Traffic, target, machine size	**Cited** (section 5)
Idle TTFT per request	**Our fit** to two published medians
Real GPU latency	**Not measured.** Nothing here ran on a GPU
Batching	**Left out.** A machine does one first token at a time; real servers batch, and batching changes prefill time

	Status
Network time, decode	**Left out.** Only queueing plus prefill
Fleet size	**Not claimed.** The traces are samples of two services; the answers are for this model of these samples

The part that carries over unchanged is the search: plug in a real load test as the check, and it makes the same calls.

7. Tests

Area	What they protect
Search	the section 2 row check by check; edges of the range; no candidate checked twice; at most log2 checks; 4,000 random cases against trying every n; no overflow near `int.MaxValue`; cancellation
Doubling	its path; cost depends on the answer, not the bound; never above the limit; "no answer" when nothing passes
Simulator	small cases you can do on paper; waiting; ties; nearest-rank percentile; monotone per request; more machines than requests changes nothing
Slowdown target	TTFT over idle TTFT; a short prompt stuck behind a long one; only N to Y as machines are added
Trace and line	CSV parsing and bad rows; arrivals relative to the first request; busiest second; the line gives back Splitwise's four medians

8. Run it

The trace files aren't in this repo. Download `AzureLLMInferenceTrace_code.csv` and `AzureLLMInferenceTrace_conv.csv` from the dataset page.

```
cd dsa/search/fewest-gpus-for-slo
dotnet test
dotnet run --project demo/FewestGpus.Demo -- --trace <path>/AzureLLMInferenceTrace_code.csv --gpu A100
```

Requires .NET 10 SDK.

9. Common mistakes

Mistake	What goes wrong
Not checking the check is monotone	Binary search returns a wrong answer, silently
`(lo + hi) / 2`	Overflows on large ranges; use `lo + (hi - lo) / 2`
A safe but huge upper bound	More checks than trying every n (section 5)
Sizing on average traffic	Bursts break the target
Returning the top of the range when nothing passes	Claims n works when it doesn't

Sources

- Google, *Site Reliability Engineering*, ch. 4: Service Level Objectives
- NVIDIA NIM: LLM benchmarking metrics
- NVIDIA Triton: Perf Analyzer CLI
- Patel et al. (2024), "Splitwise: Efficient generative LLM inference using phase splitting", ISCA: arXiv:2311.18677
- Microsoft Azure: Azure LLM Inference Dataset 2023, CC-BY. Data not included here.